



**UNIVERSIDADE VEIGA DE ALMEIDA
CURSOS DE ENGENHARIA DA COMPUTAÇÃO E CIÊNCIA DA
COMPUTAÇÃO**

Renan Martins Pereira - 1240110606

Vitor Antônio Henriques Negreiros - 1240109245

Gustavo Rangel Valente - 1240110185

Cauã Manuel Proença de Andrade -1240109764

RABALTHO A4 – Desenvolvimento de Software
LUDOTECA – BIBLIOTECA DE JOGOS DIGITAIS

Rio de Janeiro/RJ
2026

Renan Martins Pereira - 1240110606

Vitor Antônio Henriques Negreiros - 1240109245

Gustavo Rangel Valente - 1240110185

Cauã Manuel Proença de Andrade -1240109764

TRABALHO A4
LUDOTECA – BIBLIOTECA DE JOGOS DIGITAIS

LINK GIT:

<https://github.com/renanzmp/ludoteca-web>

Trabalho A4 apresentado para cumprimento do componente curricular Laboratório de Desenvolvimento de Software dos Cursos de Engenharia da Computação e Ciência da Computação da Universidade Veiga de Almeida.

Orientador: Paulo Marcos Figueiredo de Andrade

Rio de Janeiro/RJ
2026

LISTA DE TABELAS

Tabela 1 – Regras de Negócio

Tabela 2 – Requisitos Funcionais

Tabela 3 – Requisitos Não Funcionais

SUMÁRIO

1. INTRODUÇÃO	4	6
2. Fundamentação Teórica		6
2.1 Engenharia de Software		6
2.2 Metodologias Ágeis (Scrum e Kanban)		6
2.3 Arquitetura em Camadas (Controller, Service, Repository)		7
2.4 Padrão MVT do Django		7
2.5 Princípios SOLID		7
2.6 Padrões de Projeto GoF (Factory, Singleton e Strategy)		7
2.7 API REST		7
2.8 Python		7
2.9 Django		7
2.10 Django REST Framework (DRF)		8
2.11 Banco de Dados Relacional (SQL) e ORM		8
2.12 Autenticação e Segurança		8
3. Desenvolvimento do Sistema		8
3.1 Objetivo		8
3.2 Arquitetura		8
3.3 Estrutura do Projeto		8
3.4 Banco de Dados		8
3.5 Backend		8
3.6 Frontend		9
3.7 Regras de Negócio		9
3.8 Requisitos Funcionais		9
3.9 Requisitos Não Funcionais		10
4. Documentação das Telas		10
4.1 Tela de Login		10
4.2 Tela de Cadastro		11
4.3 Página Inicial e Filtros		11
4.4 Tela de Detalhes do Pet – Sem Login		Erro! Indicador não definido.
4.5 Página Inicial (Usuário Autenticado)		Erro! Indicador não definido.
4.6 ABA de Notificações		Erro! Indicador não definido.
4.7 Tela Quem Somos		Erro! Indicador não definido.
4.8 Tela de detalhes do Pet – Com Login		Erro! Indicador não definido.
Logo em seguida aparece a forma de contato com a Dona		Erro! Indicador não definido.

Ou se tentar mais de uma vez aparece que já foi marcado..**Erro! Indicador não definido.**

5. Organização Ágil e Backlog	12
6. Versionamento	13
7. Resultados	13
8. Conclusão	13
LINK – GIT	13
9. Ideias de melhorias	14
REFERÊNCIAS.....	14

1. INTRODUÇÃO

A evolução contínua da tecnologia da informação modificou profundamente as interações humanas e os processos cotidianos, impulsionando a digitalização de serviços e o surgimento de plataformas web robustas. A Engenharia de Software atua como a base estrutural desse cenário, entregando diretrizes e padrões que asseguram a criação de sistemas confiáveis, protegidos e de fácil escalabilidade.

No contexto do entretenimento digital, colecionadores e entusiastas frequentemente esbarram na dificuldade de manter um inventário organizado de seus jogos. O controle dessas bibliotecas acaba sendo fragmentado em planilhas manuais ou em ferramentas genéricas, o que compromete a privacidade e a eficiência na busca por informações específicas.

Para preencher essa lacuna, foi desenvolvida a Ludoteca Web, uma plataforma focada no gerenciamento isolado e privativo de catálogos de jogos. O sistema viabiliza a criação de contas, a autenticação blindada e a manutenção completa do acervo (criação, visualização, modificação e deleção de títulos), entregando ao usuário um ambiente seguro e exclusivo.

Alinhado às exigências modernas da disciplina, o projeto adotou o ecossistema Python em sua essência. O núcleo lógico (backend) foi construído utilizando Django e Django REST Framework, encarregados de orquestrar as regras de negócios e prover uma API RESTful. A camada de dados opera sobre um banco relacional (SQLite localmente e PostgreSQL na nuvem), gerenciado pelo ORM integrado do Django. Para a interface gráfica, optou-se pela biblioteca React com o empacotador Vite, proporcionando fluidez de navegação.

O desenvolvimento foi pautado por práticas consolidadas de engenharia, como a divisão arquitetural em camadas, adoção de preceitos SOLID e padrões GoF. Todo o fluxo de trabalho foi sustentado por métodos ágeis e versionamento colaborativo no GitHub. Este documento relata em detalhes os fundamentos, a estruturação e os desfechos obtidos com a construção e o deploy da aplicação.

2. Fundamentação Teórica

2.1 Engenharia de Software

Trata-se de uma vertente da computação voltada à aplicação de processos disciplinados e sistemáticos para a concepção de programas. Seu escopo vai muito além da escrita de código, englobando a coleta de requisitos, o desenho arquitetural, a validação por testes e o suporte a longo prazo, garantindo a entrega de uma solução de alta coesão.

2.2 Metodologias Ágeis (Scrum e Kanban)

São frameworks de gestão que privilegiam a flexibilidade e a entrega contínua de valor. O Scrum fatia o projeto em ciclos curtos denominados *sprints*, guiados por uma lista de prioridades (*backlog*). O Kanban apoia esse fluxo de forma

visual, utilizando quadros dinâmicos para mapear o status das tarefas em tempo real, evitando sobrecarga de trabalho

2.3 Arquitetura em Camadas (Controller, Service, Repository)

É um modelo estrutural que promove o desacoplamento do código. Os *Controllers* interceptam os chamados externos, os *Services* executam a lógica central de negócios, e os *Repositories* lidam exclusivamente com a persistência e resgate de dados, facilitando a substituição de componentes sem quebrar o sistema.

2.4 Padrão MVT do Django

O ecossistema Django adota uma variação arquitetural conhecida como MVT. O *Model* cuida do mapeamento dos dados no banco, a *View* orchestra a lógica de processamento HTTP, e o *Template* (substituído pelo React neste projeto) cuida da apresentação final.

2.5 Princípios SOLID

O ecossistema Django adota uma variação arquitetural conhecida como MVT. O *Model* cuida do mapeamento dos dados no banco, a *View* orchestra a lógica de processamento HTTP, e o *Template* (substituído pelo React neste projeto) cuida da apresentação final.

2.6 Padrões de Projeto GoF (Factory, Singleton e Strategy)

Consistem em catálogos de soluções universais para dilemas frequentes de modelagem. A aplicação correta de padrões como *Factory* (para instanciar objetos complexos) ou *Singleton* (para garantir instâncias únicas) padroniza e otimiza o código-fonte.

2.7 API REST

É um padrão de comunicação leve e *stateless* que utiliza a semântica nativa do HTTP para transacionar dados (geralmente em JSON). Essa abordagem descentraliza o processamento e permite que o servidor atenda a requisições de forma independente e padronizada.

2.8 Python

Linguagem pragmática, valorizada pela sua curva de aprendizado acessível e sintaxe expressiva. Suas vastas bibliotecas a tornam uma escolha soberana para a construção de backends complexos.

2.9 Django

Um framework web *batteries-included* que acelera o desenvolvimento em Python ao fornecer soluções embutidas para roteamento, mapeamento de banco de dados e mitigação de vulnerabilidades cibernéticas.

2.10 Django REST Framework (DRF)

Ferramenta complementar ao Django que simplifica abruptamente a elaboração de Web APIs. Ela introduz mecanismos poderosos de serialização e políticas ricas para controle de acessos.

2.11 Banco de Dados Relacional (SQL) e ORM

Bancos relacionais armazenam registros em tabelas interconectadas, preservando a integridade das informações. O Mapeador Objeto-Relacional (ORM) traduz essas estruturas para classes da linguagem de programação, eliminando a necessidade de escrever comandos SQL manualmente.

2.12 Autenticação e Segurança

Mecanismo vital para a proteção de rotas privadas. O sistema adota os *JSON Web Tokens* (JWT) para endossar a identidade dos usuários de maneira segura, enquanto as senhas são permanentemente criptografadas antes do armazenamento.

3. Desenvolvimento do Sistema

3.1 Objetivo

Esta seção mapeia os componentes estruturais da Ludoteca Web, detalhando a mecânica sob a qual a aplicação foi concebida, desde o repositório de dados até as interfaces interativas.

3.2 Arquitetura

A solução adota uma topologia cliente-servidor descentralizada. O navegador atua como cliente consumindo a interface React, comunicando-se ativamente com a API gerida pelo Django, que, por sua vez, centraliza as regras de negócio e a validação do banco de dados.

3.3 Estrutura do Projeto

O código base é segmentado em duas pastas autônomas: o *frontend* (gerido pelo Vite) e o *backend* (contendo as *apps* do Django). Isso permitiu implantar as camadas em servidores distintos sem atritos logísticos.

3.4 Banco de Dados

O armazenamento é sustentado por arquitetura relacional, ancorada pelo SQLite na fase de codificação e elevada ao PostgreSQL para o ambiente em nuvem. A modelagem abrange os usuários e suas respectivas coleções de jogos, blindando os dados por meio de chaves de isolamento (*multi-tenancy*).

3.5 Backend

Núcleo processador da plataforma. Programado com Python/DRF, ele se encarrega das validações, do controle de permissões por JWT e das manobras

operacionais de CRUD (Criação, Leitura, Atualização e Deleção) sobre o acervo de cada indivíduo.

3.6 Frontend

A vitrine do usuário. Edificada com React, ela substitui a recarga tradicional de páginas HTML por uma experiência fluida de *Single Page Application*, adotando interfaces escuras e limpas para facilitar a navegação.

3.7 Regras de Negócio

As premissas operacionais que regulam a lógica do produto estão documentadas na Tabela 1.

Tabela 1 – Regras de Negócio

Código	Descrição
RN01	Todo catálogo é estritamente privado; um perfil não possui acesso ao acervo de terceiros.
RN02	O endereço de e-mail atua como a chave primária de identificação no momento do acesso.
RN03	É terminantemente proibido o registro de contas duplicadas utilizando o mesmo e-mail.
RN04	A modificação ou consulta de dados requer um token JWT validado nos cabeçalhos da requisição.
RN05	O campo "nome de exibição" não exige unicidade, permitindo cadastros de homônimos.

3.8 Requisitos Funcionais

As tarefas primárias que o sistema deve executar estão delineadas na Tabela 2.

Tabela 2 – Requisitos Funcionais

Código	Descrição
RF01	Possibilitar o registro de perfis contendo nome, e-mail e credencial secreta.
RF02	Avaliar credenciais e retornar chaves de sessão autenticadas.
RF03	Fornecer um meio para a inserção de novos jogos no banco de dados.
RF04	Apresentar um painel de visualização exclusiva dos títulos salvos pelo usuário logado.
RF05	Viabilizar a correção textual de atributos de um jogo já inserido.
RF06	Autorizar a supressão permanente de um item do catálogo.

3.9 Requisitos Não Funcionais

As exigências técnicas relativas ao desempenho e à infraestrutura constam na Tabela 3.

Tabela 3 – Requisitos Não Funcionais

Código	Descrição
RNF01	O núcleo de processamento deve ser estabelecido com a linguagem Python (Django).
RNF02	O armazenamento em nuvem será suportado pelo PostgreSQL.
RNF03	A camada visual deve ser manipulada pelo framework React.
RNF04	O servidor precisa restringir requisições cruzadas (CORS) apenas para as origens autorizadas.
RNF05	O sistema deve seguir arquitetura em camadas e os princípios SOLID.
RNF06	O controle das alterações do código-fonte será realizado pela ferramenta Git.
RNF07	A aplicação deve possuir documentação da API.

4. Documentação das Telas

Esta seção apresenta as principais telas do sistema Get a Pet, descrevendo brevemente a função de cada uma.

4.1 Tela de Login

Ponto de partida para utilizadores recorrentes. A interface requer e-mail e senha, processando alertas imediatos caso a combinação fornecida não conste na base de dados.

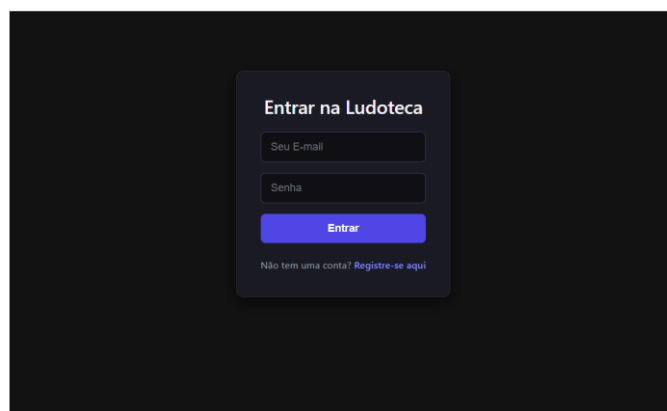


Figura 1 – Tela de Login

4.2 Tela de Cadastro

A tela de cadastro permite a criação de uma nova conta por meio do preenchimento de nome completo, e-mail, telefone, senha e confirmação de senha, com validação dos campos obrigatórios.

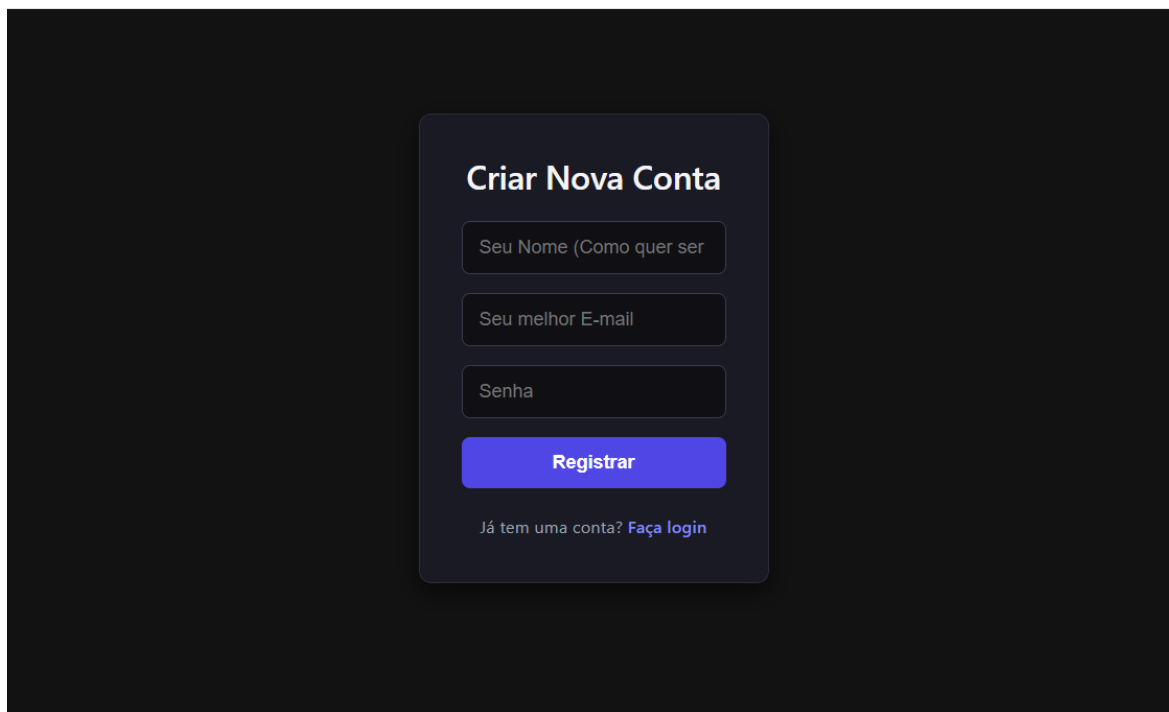
A imagem mostra a tela de cadastro de uma aplicação web. No topo, há o título "Criar Nova Conta" em uma fonte branca sobre um fundo escuro. Abaixo do título, há três campos de entrada de texto empilhados verticalmente: "Seu Nome (Como quer ser)", "Seu melhor E-mail" e "Senha". Cada campo tem uma borda arredondada e um ícone de lupa no canto superior direito. Abaixo dos campos, há um botão azul com o texto "Registrar" em branco. Na base da seção, há um link de texto que diz "Já tem uma conta? Faça login".

Figura 2 – Tela de Cadastro

4.3 Página Inicial e Dashboard

Ambiente central da plataforma. Uma vez autenticado, o usuário visualiza uma grade interativa (*cards*) com todos os seus jogos, além de um formulário simplificado para o cadastramento imediato de novos itens.

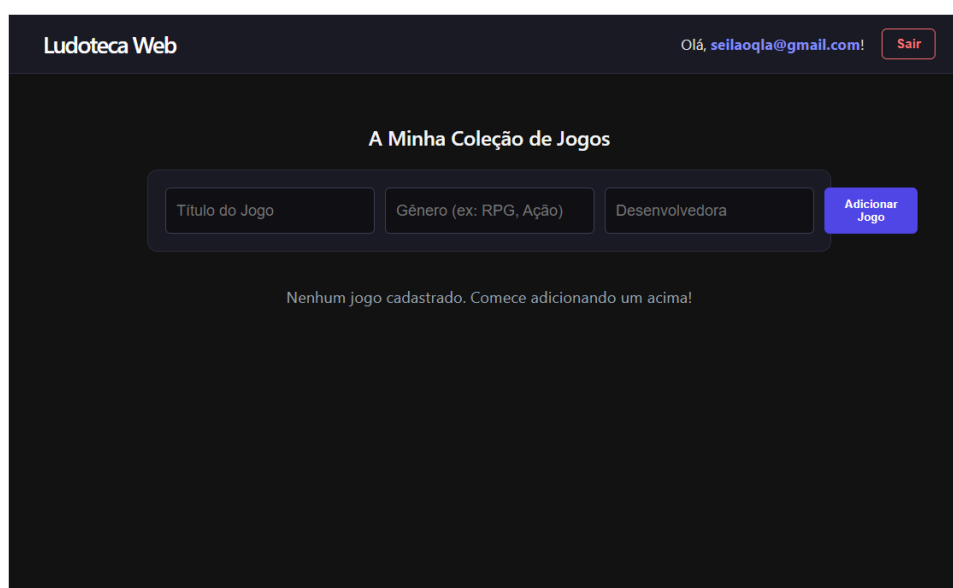
A imagem mostra a interface de usuário da "Ludoteca Web". No topo, há uma barra de navegação com o nome "Ludoteca Web" à esquerda, o texto "Olá, seilaoqla@gmail.com!" no centro e um botão "Sair" à direita. Abaixo, há uma seção intitulada "A Minha Coleção de Jogos". Nesta seção, há um formulário com três campos de entrada: "Título do Jogo", "Gênero (ex: RPG, Ação)" e "Desenvolvedora". À direita desses campos, há um botão azul "Adicionar Jogo". Abaixo do formulário, há uma mensagem de texto que diz "Nenhum jogo cadastrado. Comece adicionando um acima!".

Figura 3 – Página Inicial sem jogo adicionado

4.4 Interação e Edição de Jogos

Ao navegar pelos seus jogos, o utilizador encontra comandos intuitivos em cada *card* para ativar os modos de alteração textual ou solicitar a exclusão definitiva do registro.

Ludoteca Web

Olá, seilaoqla@gmail.com! [Sair](#)

A Minha Coleção de Jogos

Título do Jogo Gênero (ex: RPG, Ação) Desenvolvedora [Adicionar Jogo](#)

The Last of US
Ação • Naughty Dog [Editar](#) [Remover](#)

Mortal Kombat 1
Luta • NetherRealm [Editar](#) [Remover](#)

FIFA
Futebol • EA sports [Editar](#) [Remover](#)

Figura 4 – Tela com opções de editar e remover

5. Organização Ágil e Backlog

A trajetória de desenvolvimento obedeceu aos ciclos propostos pelas metodologias ágeis. As entregas foram particionadas em histórias de usuário e geridas gradualmente. O escopo mínimo operável foi delineado na tabela abaixo.

Backlog do Produto

ID	História de Usuário	Prioridade	Status
US01	Como visitante, quero criar um perfil para usufruir da plataforma.	Alta	Concluído
US02	Como usuário, almejo adicionar títulos à minha coleção pessoal.	Alta	Concluído
US03	Como usuário, espero que o sistema blinde o meu acervo contra o acesso de outras pessoas.	Alta	Concluído
US04	Como usuário, necessito corrigir informações que digitei erroneamente.	Média	Concluído

US05	Como usuário, exijo a opção de descartar itens do meu catálogo.	Média	Concluído
US06	Como programador, almejo orquestrar a API em um servidor remoto de produção.	Alta	Concluído
US07	Como programador, desejo integrar o frontend em nuvem com o banco de dados definitivo.	Alta	Concluído

6. Versionamento

Todo o ecossistema do projeto foi devidamente versionado através do sistema Git, mantendo o repositório hospedado e sincronizado no GitHub. Essa diretriz não só preservou o histórico das modificações como também viabilizou a entrega contínua (CI/CD) ao conectar os ramos de código com os servidores de *deploy* (Vercel e Render).

7. Resultados

A conclusão das etapas de engenharia resultou na entrega integral da Ludoteca Web. A plataforma é plenamente capaz de suportar operações de cadastro, gerenciamento de tokens e manipulações de registros sob a premissa fundamental da privacidade (*multi-tenancy*).

A fase de hospedagem desafiou a equipe com ajustes finos de produção, envolvendo políticas restritas de CORS, ocultação de criptografias de estado e compilação de arquivos estáticos, barreiras que foram completamente vencidas para a estabilização do serviço.

8. Conclusão

A execução desta plataforma proporcionou a consolidação técnica sobre a construção e o provisionamento de arquiteturas web separadas. O intercâmbio entre React, Vite, Django e PostgreSQL atesta a necessidade de um desenho de software coeso e seguro. Para além das entregas exigidas, o processo reforçou a relevância do controle ágil e do versionamento. A Ludoteca Web apresenta-se hoje não apenas como uma ferramenta funcional, mas como um esqueleto tecnológico perfeitamente desenhado para adaptações e expansões comerciais futuras.

LINK – GIT

<https://github.com/renanzmp/ludoteca-web>

9. Ideias de melhorias

Consumo automático de dados através da API pública da IGDB para obtenção de resumos e imagens artísticas dos jogos.

Módulo para envio e hospedagem de imagens de capa personalizadas na nuvem.

Adição de sistema de pontuações e status de progressão (ex: "Zerado", "Jogando", "Na fila").

Habilitar uma chave pública seletiva para que o usuário possa divulgar um link de exibição de sua coleção.

Fluxo de resgate de acesso por meio do envio de tokens para recuperação de senhas por e-mail.

REFERÊNCIAS

DJANGO SOFTWARE FOUNDATION. Django documentation. Disponível em: <https://docs.djangoproject.com/>. Acesso em: jul. 2026.

DJANGO REST FRAMEWORK. Home. Disponível em: <https://www.django-rest-framework.org/>. Acesso em: jul. 2026.

PYTHON SOFTWARE FOUNDATION. Python documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: jul. 2026.

REACT. React Documentation. Disponível em: <https://react.dev/>. Acesso em: jul. 2026.

RENDER. Render Cloud Application Hosting. Disponível em: <https://render.com/docs>. Acesso em: jul. 2026.

VERCEL. Next.js and Vite Deployment Documentation. Disponível em: <https://vercel.com/docs>. Acesso em: jul. 2026.